JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class SimpleFormatter

java.lang.Object
    java.util.logging.Formatter
        java.util.logging.SimpleFormatter

```
public class SimpleFormatter
extends Formatter
```

Print a brief summary of the `LogRecord` in a human readable format. The summary will typically be 1 or 2 lines.

Configuration: The `SimpleFormatter` is initialized with the format string specified in the `java.util.logging.SimpleFormatter.format` property to format the log messages. This property can be defined in the logging properties configuration file or as a system property. If this property is set in both the logging properties and system properties, the format string specified in the system property will be used. If this property is not defined or the given format string is illegal, the default format is implementation-specific.

Since:

1.4

See Also:

`Formatter`

Constructor Summary

Constructors

Constructor	Description
`SimpleFormatter()`	Create a `SimpleFormatter`.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`String`	`format(LogRecord record)`	Format the given LogRecord.

Methods declared in class Formatter

`formatMessage`, `getHead`, `getTail`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

SimpleFormatter

`public SimpleFormatter()`

Create a `SimpleFormatter`.

Method Details

format

`public String format(LogRecord record)`

Format the given LogRecord.

The formatting can be customized by specifying the format string in the `java.util.logging.SimpleFormatter.format` property. The given `LogRecord` will be formatted as if by calling:

```
String.format(format, date, source, logger, level, message, thrown);
```

where the arguments are:

1. `format` - the `java.util.Formatter` format string specified in the `java.util.logging.SimpleFormatter.format` property or the default format.
2. `date` - a `ZonedDateTime` object representing event time of the log record in the `ZoneId.systemDefault()` system time zone.
3. `source` - a string representing the caller, if available; otherwise, the logger's name.
4. `logger` - the logger's name.
5. `level` - the log level.
6. `message` - the formatted log message returned from the `Formatter.formatMessage(LogRecord)` method. It uses `java.text` formatting and does not use the `java.util.Formatter format` argument.
7. `thrown` - a string representing the throwable associated with the log record and its backtrace beginning with a newline character, if any; otherwise, an empty string.

Some example formats:

- `java.util.logging.SimpleFormatter.format="%4$s: %5$s [%1$tc]%n"`

  This prints 1 line with the log level (4$), the log message (5$) and the timestamp (1$) in a square bracket.

  ```
  WARNING: warning message [Tue Mar 22 13:11:31 PDT 2011]
  ```

- `java.util.logging.SimpleFormatter.format="%1$tc %2$s%n%4$s: %5$s%6$s%n"`

  This prints 2 lines where the first line includes the timestamp (1$) and the source (2$); the second line includes the log level (4$) and the log message (5$) followed by the throwable and its backtrace (6$), if any:

  ```
  Tue Mar 22 13:11:31 PDT 2011 MyClass fatal
  SEVERE: several message with an exception
  java.lang.IllegalArgumentException: invalid argument
          at MyClass.mash(MyClass.java:9)
          at MyClass.crunch(MyClass.java:6)
          at MyClass.main(MyClass.java:3)
  ```

- `java.util.logging.SimpleFormatter.format="%1$tb %1$td, %1$tY %1$tl:%1$tM:%1$tS %1$Tp %2$s%n%4$s: %5$s%n"`

  This prints 2 lines similar to the example above with a different date/time formatting and does not print the throwable and its backtrace:

  ```
  Mar 22, 2011 1:11:31 PM MyClass fatal
  SEVERE: several message with an exception
  ```

- `java.util.logging.SimpleFormatter.format="%1$tb %1$td, %1$tY %1$tl:%1$tM:%1$tS.%1$tN %1$Tp %2$s%n%4$s: %5$s%6$s%n"`

  Since JDK 9, `java.util.logging` uses `java.time` to create more precise time stamps. The format above can be used to add a `.%1$tN` to the date/time formatting so that nanoseconds will also be printed:

```
Feb 06, 2015 5:33:10.279216000 PM example.Main main
INFO: This is a test
```

This method can also be overridden in a subclass. It is recommended to use the `Formatter.formatMessage(LogRecord)` convenience method to localize and format the message field.

Specified by:

`format` in class `Formatter`

Parameters:

`record` - the log record to be formatted.

Returns:

a formatted log record

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.